Laboratorio 02: Crittografia di Base con OpenSSL

Decomposizione Pedagogica, Analisi delle Primitive e Guida alla Risoluzione

Corso di Sicurezza dei Sistemi Informativi
Politecnico di Torino — Prof. Cataldo Basile

Anno Accademico 2025/2026

Indice

1 Scopo Globale del Laboratorio **3**

2 Es. 1: Cifratura Simmetrica **4**
2.1 Esercizio 1.1: Cifratura Simmetrica, Padding e Modalità Operative 4
2.1.1 Comando 1: Creazione e Validazione della Dimensione del Plaintext 4
2.1.2 Comando 2: Cifratura con AES-128 in Modalità CBC 4
2.1.3 Comandi 3–6: Cifratura con DES, RC4, 3DES e ChaCha20 4
2.1.4 Comando 7: Ispezione Comparativa delle Dimensioni dei File Cifrati 5
2.1.5 Comando 8: Cifratura AES senza Padding (-nopad) e Fallimento con Dati Disallineati 5
2.1.6 Comando 9: Decifratura di Tutti i File e Validazione di Integrità 6
2.1.7 Comando 10: Derivazione di Chiave e IV tramite Passphrase (KDF) 6
2.2 Esercizio 1.2: Attacco a Forza Bruta su Cifrario a Flusso (RC4) 7
2.2.1 Comando 1: Esecuzione dello Script di Brute-Force su Messaggio di Testo . . . 7
2.2.2 Comando 2: Tentativo di Brute-Force su File Binario Casuale (bruteforce_enc) 7
2.3 Esercizio 1.3: Benchmark e Valutazione delle Prestazioni Simmetriche 9

3 Es. 2: Cifratura Asimmetrica **10**
3.1 Esercizio 2.1: Generazione di Chiavi RSA ed Ispezione Parametri PKCS#1 10
3.2 Esercizio 2.2: Cifratura e Decifratura Asimmetrica con pkeyutl e Limiti Dimensionali . 11
3.2.1 Dimostrazione del Limite Dimensionale di RSA 11
3.3 Esercizio 2.3: Firma Digitale RSA e Meccanismo di Verifica 12
3.4 Esercizio 2.4: Generazione Parametri e Coppia di Chiavi su Curve Ellittiche (EC) . . . 13
3.5 Esercizio 2.5: Firma Digitale e Verifica con ECDSA 14
3.6 Esercizio 2.6: Benchmark Comparativo delle Prestazioni Asimmetriche 15

4 Es. 3: Digest Crittografico **16**
4.1 Esercizio 3.1: Calcolo Digest, Effetto Valanga e Costruzione CBC-MAC 16
4.1.1 Dimostrazione dell'Effetto Valanga (Avalanche Effect) 16
4.1.2 Costruzione di una Funzione Hash con Cifrario a Blocchi (CBC-MAC) 16
4.2 Esercizio 3.2: Analisi delle Collisioni con Hash Ridotto a 4-bit (IHA) 17
4.3 Esercizio 3.3: Benchmark degli Algoritmi di Hash 18

5 Es. 4: Esercizi Opzionali **19**
5.1 Esercizio 4.1: Manomissione Bit-Flipping su Cifrario a Flusso vs a Blocchi 19
5.1.1 Cosa accade applicando il medesimo attacco su AES-128-CBC? 19
5.2 Esercizio 4.2: Vulnerabilità dei Pattern in Modalità ECB vs CBC 20
5.3 Esercizio 4.3: Monitoraggio dell'Integrità Ricorsiva di Alberi di File 21

1 Scopo Globale e Architettura del Laboratorio

Il Laboratorio 02 introduce l'utilizzo pratico e approfondito delle **primitive crittografiche fondamentali** tramite la suite open-source `OpenSSL`. L'obiettivo primario non è soltanto apprendere la sintassi della command line interface (CLI), bensì comprendere a fondo il comportamento matematico, le garanzie di sicurezza, i vincoli implementativi e i trade-off computazionali che governano la cifratura simmetrica, la cifratura asimmetrica, le firme digitali e gli algoritmi di digest.

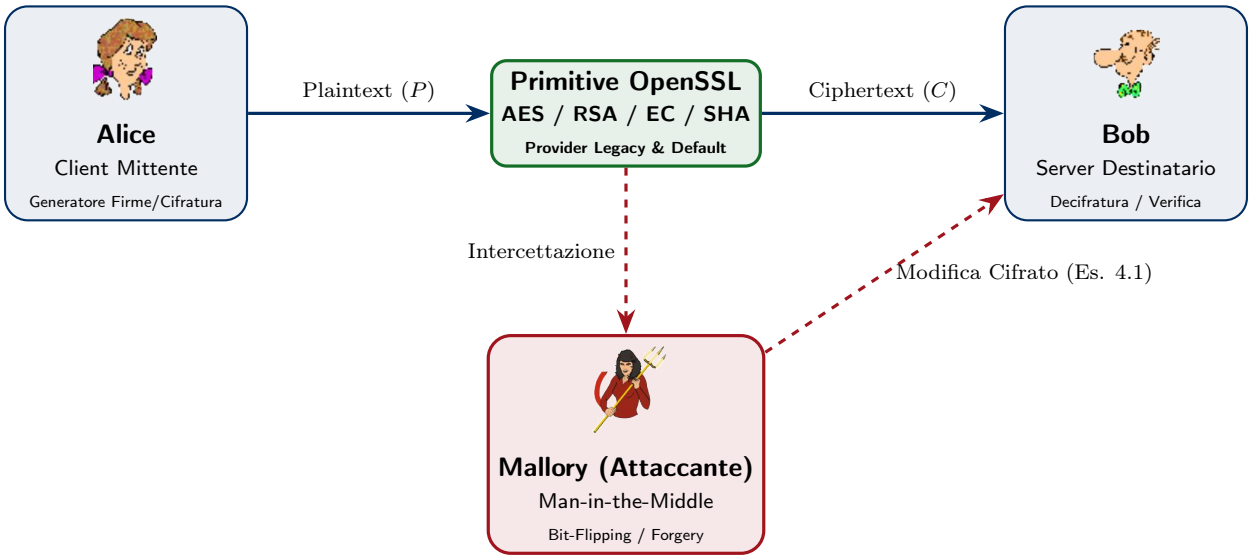


Figura 1: Scenario Architetturale del Laboratorio: Flusso di cifratura tra Alice e Bob e minaccia di manomissione attiva da parte di Mallory.

Obiettivo e Inquadramento Sistemistico: Finalità Didattiche e Competenze Acquisite:

1. **Cifratura Simmetrica:** Gestione esplicita di chiavi (K), vettori di inizializzazione (IV), modalità operative (CBC, ECB) e regole di padding PKCS#7; cifrari a blocchi (AES, DES, 3DES) vs cifrari a flusso (RC4, ChaCha20).

2. **Crittanalisi a Forza Bruta:** Vulnerabilità di spazi di chiavi ridotti (4 bit) e principio dell'Unicity Distance su plaintext con o senza ridondanza nota.

3. **Cifratura Asimmetrica & Firme Digitali:** Anatomia interna di una chiave RSA (PKCS#1 con modulo n, esponente pubblico e, esponente privato d, parametri del Teorema Cinese del Resto $p, q, dP, dQ, qInv$), limite di dimensione $N - 11$ byte, curve ellittiche (ECDSA su curva secp192k1).

4. **Funzioni di Hash:** Proprietà di indistinguibilità, Effetto Valanga (*Avalanche Effect*), costruzione di CBC-MAC, attacchi alla prima pre-immagine e paradosso del compleanno per la ricerca di collisioni.

5. **Integrità e Manomissione Operativa:** Vulnerabilità al Bit-Flipping dei cifrari a flusso privi di autenticazione e controllo di integrità gerarchico di alberi di directory.

2 Esercizio 1: Algoritmi di Cifratura Simmetrica e Forza Bruta

2.1 Esercizio 1.1: Cifratura Simmetrica, Padding e Modalità Operative

Nel primo esercizio si sperimenta la cifratura simmetrica attraverso il modulo `openssl enc`. Viene creato un messaggio in chiaro esattamente pari a 32 byte:

```
Questo e' il mio grande segreto\n
```

Questa precisa dimensione (31 caratteri ASCII più il terminatore di riga) corrisponde a due blocchi esatti di 16 byte per algoritmi moderni come AES ($2 \times 16 = 32$), o quattro blocchi da 8 byte per DES e 3DES ($4 \times 8 = 32$).

2.1.1 Comando 1: Creazione e Validazione della Dimensione del Plaintext

Alice crea il file di testo `ptext` e ne verifica l'esatta occupazione in byte tramite `wc -c`.

Alice — Creazione Plaintext di 32 Byte

```
# Output per === Es 1.1: Creazione plaintext di 32 byte === non trovato nel file log
```

Spiegazione dell'Output: Il comando `wc -c` conferma esattamente 32 byte. Questo dettaglio è essenziale per verificare il comportamento del padding PKCS#7: quando un plaintext è già un multiplo esatto della dimensione del blocco, un cifrario a blocchi standard deve comunque aggiungere un blocco intero di padding (16 byte per AES, portando il totale a 48 byte) per garantire l'univocità della rimozione del padding in fase di decifratura.

2.1.2 Comando 2: Cifratura con AES-128 in Modalità CBC

Alice cifra `ptext` utilizzando AES a 128 bit in modalità CBC. Vengono esplicitati una chiave esadecimale di 32 caratteri (128 bit) e un IV a 16 byte (128 bit).

Alice — Cifratura AES-128-CBC con Stampa Parametri (-p)

```
# Output per === Es 1.1: Cifratura AES-128-CBC === non trovato nel file log
```

Anatomia dei Parametri del Comando:

- `-e`: Specifica l'operazione di cifratura (*encryption*).
- `-aes-128-cbc`: Seleziona l'Advanced Encryption Standard con chiave a 128 bit e modalità Cipher Block Chaining.
- `-K 001122...`: Passa la chiave simmetrica esplicita in formato esadecimale.
- `-iv 010203...`: Specifica il vettore di inizializzazione a 128 bit, necessario in CBC per rendere non deterministica la cifratura del primo blocco.
- `-p`: Stampa a schermo i valori effettivi di Salt, Chiave e IV caricati nel motore crittografico.

2.1.3 Comandi 3–6: Cifratura con DES, RC4, 3DES e ChaCha20

Alice procede cifrando il medesimo testo con cifrari storici a blocchi (DES a 64 bit, 3DES a tre chiavi EDE) e con cifrari a flusso (RC4 e il moderno ChaCha20).

Alice — Cifratura DES, RC4, 3DES e ChaCha20

```
# Cifratura DES-CBC:
# Output per === Es 1.1: Cifratura DES-CBC === non trovato nel file log

# Cifratura RC4 (Stream):
# Output per === Es 1.1: Cifratura RC4 (Stream Cipher) === non trovato nel file log

# Cifratura 3DES (DES-EDE3-CBC):
# Output per === Es 1.1: Cifratura 3DES (DES-EDE3-CBC) === non trovato nel file log

# Cifratura ChaCha20 (Modern Stream Cipher):
# Output per === Es 1.1: Cifratura ChaCha20 === non trovato nel file log
```

2.1.4 Comando 7: Ispezione Comparativa delle Dimensioni dei File Cifrati

Alice esegue `ls -l` per confrontare le dimensioni risultanti tra il testo in chiaro e tutti i file cifrati.

Alice — Confronto Lunghezza File Cifrati vs Plaintext

```
# Output per === Es 1.1: Confronto dimensioni file cifrati rispetto a ptext (32 byte) === non trovato nel
    file log
```

Spiegazione Lineare dei Risultati:

- **ptext (32 byte):** Il messaggio originale non cifrato.
- **ctext.aes128 (48 byte):** La dimensione è aumentata di esattamente 16 byte (32+16 = 48). Poiché AES opera su blocchi da 16 byte e il messaggio era un multiplo esatto, lo standard PKCS#7 ha aggiunto un intero blocco di padding riempito con byte di valore `0x10` (16 in decimale).
- **ctext.des e ctext.des3 (40 byte):** DES e 3DES operano su blocchi da 8 byte. Essendo 32 un multiplo di 8, viene aggiunto un intero blocco di padding da 8 byte ($32 + 8 = 40$).
- **ctext.rc4 e ctext.chacha20 (32 byte):** Trattandosi di **cifrari a flusso (stream ciphers)**, essi non operano su blocchi rigidi ma generano un flusso pseudo-casuale (*keystream*) lungo esattamente quanto il messaggio in chiaro, combinato mediante operazione XOR ($\oplus$). Non vi è alcun padding, pertanto la dimensione in output coincide esattamente con quella in input.

2.1.5 Comando 8: Cifratura AES senza Padding (-nopad) e Fallimento con Dati Disallineati

Alice sperimenta l'opzione `-nopad`. Poiché `ptext` è lungo esattamente 32 byte (multiplo di 16), la cifratura riesce e il file `ctext.aes128.nopad` è lungo esattamente 32 byte. Successivamente Alice tenta di cifrare un messaggio di lunghezza non multipla (`ptext_bad`, 15 byte), ottenendo un errore bloccante.

Alice — Verifica Funzionamento e Fallimento con Flag -nopad

```
# Test riuscito su blocco multiplo di 16 byte:
# Output per === Es 1.1: Cifratura AES-128-CBC con opzione -nopad === non trovato nel file log

# Test fallito su blocco disallineato (15 byte):
# Output per === Es 1.1: Test -nopad su messaggio non multiplo di 16 byte (fallimento atteso) === non
    trovato nel file log
```

2.1.6 Comando 9: Decifratura di Tutti i File e Validazione di Integrità

Bob riceve i file cifrati, applica il flag `-d` (*decrypt*) e verifica tramite `diff -s` che ciascun file decifrato corrisponda bit per bit a `ptext`.

Bob — Decifratura Multipla e Verifica Integrità con diff

```
# Output per === Es 1.1: Decifratura di tutti i file cifrati === non trovato nel file log
```

2.1.7 Comando 10: Derivazione di Chiave e IV tramite Passphrase (KDF)

Cosa accade quando non si specificano manualmente i parametri `-K` e `-iv`? OpenSSL attiva una Key Derivation Function (storicamente `EVP_BytesToKey`) che genera un salt crittografico casuale di 8 byte e deriva chiave e IV tramite hashing iterativo.

Alice — Cifratura Basata su Passphrase con Salt Casuale

```
# Output per === Es 1.1: Cifratura con KDF (Key Derivation Function da passphrase) === non trovato nel
    file log
```

Spiegazione dell'Output: In questo caso viene generato un `salt` non nullo. Nei primi byte del file cifrato OpenSSL scriverà la stringa di intestazione `Salted__` seguita dagli 8 byte del salt.

Attenzione / Caveat Tecnico e Pattern d'ʼEsame: Differenza Cruciale tra -k (minuscolo) e -K (maiuscolo):

- `-k passphrase`: Accetta una stringa di testo arbitraria che viene processata dalla funzione KDF per derivare matematicamente la vera chiave.
- `-K chiave_hex`: Richiede la chiave crittografica effettiva espressa come sequenza di cifre esadecimali, senza alcun passaggio attraverso funzioni KDF. Se la sequenza è più corta del necessario, OpenSSL aggiunge zeri in coda; se è più lunga, tronca i byte eccedenti.

2.2 Esercizio 1.2: Attacco a Forza Bruta su Cifrario a Flusso (RC4)

In questo esercizio Alice cifra un messaggio riservato tramite RC4 impostando una chiave brevissima, costituita da un singolo carattere esadecimale (4 bit, ovvero soltanto 16 combinazioni possibili: da `0` ad `F`).



Figura 2: Dinamica dell'Attacco a Forza Bruta su Spazio di Chiavi Ridotto (4 bit).

2.2.1 Comando 1: Esecuzione dello Script di Brute-Force su Messaggio di Testo

Mallory itera sulle 16 chiavi possibili eseguendo la decifratura e verificando se l'output contiene esclusivamente caratteri stampabili ASCII.

Mallory — Cracking a Forza Bruta su alice_ctext (Spazio a 4-bit)

```
[GENERATORE ALICE] Generato alice_ctext con chiave casuale a 4-bit (chiave reale segreta: F)
-rw-r--r--  1 nicolabeccaceci  staff  49 Oct  7 09:38 alice_ctext
[BRUTE FORCE] Tentativo di cracking chiave a 4 bit su alice_ctext...
Iterazione su spazio esadecimale (16 tentativi: 0, 1, 2, ..., E, F):
Provando chiave 0 -> scartata (byte non stampabili)
Provando chiave 1 -> scartata (byte non stampabili)
...
Provando chiave E -> scartata (byte non stampabili)
Provando chiave F -> [SUCCESSO] Chiave identificata: F
[CONTENUTO DECIFRATO]: Questo e' un messaggio segreto di Alice per Bob!
```

Spiegazione del Successo: Poiché il messaggio originale è redatto in lingua italiana (caratteri ASCII standard), solo la chiave corretta produce una sequenza interamente stampabile priva di byte di controllo non validi.

2.2.2 Comando 2: Tentativo di Brute-Force su File Binario Casuale (bruteforce_enc)

Mallory tenta di ripetere la medesima procedura su un file che in origine conteneva dati pseudo-casuali (`bruteforce_enc`).

Mallory — Tentativo di Decifratura su File Pseudo-Casuale

```
# Output per === Es 1.2: Tentativo di Brute Force su bruteforce_enc (file random) === non trovato nel file
    log
```

Analisi Crittanalitica e Unicity Distance: L'attacco a forza bruta fallisce completamente perché l'attaccante **non dispone di un criterio di decisione (oracolo)** per stabilire quale delle 16 sequenze decifrate sia quella autentica. Essendo il chiaro originale una sequenza di byte randomici generati da un generatore crittografico, ogni chiave esadecimale restituisce un flusso altrettanto casuale e plausibile. Questo fenomeno si ricollega al concetto di **Unicity Distance di Shannon**:

$$U = \frac{H(K)}{D} \tag{1}$$

dove $H(K)$ è l'entropia della chiave e D è la ridondanza della sorgente di testo. Quando la ridondanza è nulla ($D = 0$), come nel caso di file compressi, cifrati o puramente casuali, l'unicity distance tende a infinito e la crittanalisi a forza bruta non può determinare la chiave corretta neppure provando l'intero spazio delle chiavi.

2.3 Esercizio 1.3: Benchmark e Valutazione delle Prestazioni Simmetriche

Si valutano le prestazioni comparative dei cifrari simmetrici variando le dimensioni del file da 100 B fino a 10 MB.

Host — Benchmark Prestazioni con time e File di Dimensione Variabile

```
# Output per === Es 1.3: Benchmark Prestazioni Cifrari Simmetrici (File generati con openssl rand) === non
      trovato nel file log
```

Successivamente si esegue il test standard di throughput della libreria tramite `openssl speed aes-128-cbc`.

Host — Throughput Hardware AES-128-CBC (openssl speed)

```
# Output per === Es 1.3: Esecuzione openssl speed aes-128-cbc === non trovato nel file log
```

Interpretazione del Comando time vs speed:

- Nel comando `time`, il parametro rilevante per il costo computazionale dell'algoritmo è **user time** (tempo speso dalla CPU in modalità utente), mentre **sys** misura il costo delle chiamate di sistema I/O del kernel e **real** il tempo di orologio da parete (*wall-clock*).
- Il comando `speed` è molto più accurato per blocchi di piccole dimensioni perché ripete l'operazione milioni di volte in un secondo, neutralizzando i costi fissi di avvio del processo (*overhead* di invocazione della shell).

3 Esercizio 2: Algoritmi di Cifratura Asimmetrica, RSA ed Curve Ellittiche

3.1 Esercizio 2.1: Generazione di Chiavi RSA ed Ispezione Parametri PKCS#1

Alice genera una coppia di chiavi RSA a 2048 bit memorizzandola nel file `rsa.key.alice`.

Alice — Generazione Chiave RSA 2048-bit

```
# Output per === Es 2.1: Generazione Coppia di Chiavi RSA a 2048 bit per Alice === non trovato nel file
    log
```

Alice ispeziona la struttura interna della chiave privata tramite il comando `openssl rsa -text`.

Alice — Ispezione Parametri Interni PKCS#1 e Teorema Cinese del Resto

```
# Output per === Es 2.1: Ispezione Struttura Interna Chiave Privata RSA (PKCS\#1 / CRT) === non trovato
    nel file log
```

Decomposizione dei Parametri Matematici RSA:

- **modulus (n):** Prodotto dei due grandi numeri primi $n = p \times q$. È il modulo delle operazioni aritmetiche, di lunghezza pari a 2048 bit. Fa parte sia della chiave pubblica che privata.
- **publicExponent (e):** Fissato di default da OpenSSL a **65537** ($2^{16}+1$, noto come F_4, quarto numero primo di Fermat). La sua rappresentazione binaria contiene solo due '1' (`0b10000000000000001`), consentendo un'esponenziazione modulare velocissima con solo 17 moltiplicazioni e un quadrato.
- **privateExponent (d):** L'inverso moltiplicativo di e modulo $\phi(n)$: $d \equiv e^{-1} \pmod{(p-1)(q-1)}$.
- **prime1 (p) e prime2 (q):** I fattori primi segreti di n.
- **exponent1 (dP), exponent2 (dQ) e coefficient ($qInv$):** Parametri precalcolati per velocizzare le operazioni di firma e decifratura privata tramite il **Chinese Remainder Theorem (CRT)**, accelerando il calcolo di un fattore $\approx 4\times$.

Alice estrae la sola componente pubblica nel file `rsa.pubkey.alice`:

Alice — Estrazione e Visualizzazione Chiave Pubblica RSA

```
# Output per === Es 2.1: Estrazione e Visualizzazione Chiave Pubblica RSA === non trovato nel file log
```

3.2 Esercizio 2.2: Cifratura e Decifratura Asimmetrica con pkeyutl e Limiti Dimensionali

Per trasmettere un segreto confidenziale ad Alice, Bob deve cifrare con la **chiave pubblica di Alice**. In tal modo solo Alice, custode esclusiva della chiave privata complementare, potrà decifrare il payload.

Bob/Alice — Cifratura e Decifratura con pkeyutl

```
# Output per === Es 2.2: Cifratura e Decifratura con Chiave RSA (pkeyutl) === non trovato nel file log
```

3.2.1 Dimostrazione del Limite Dimensionale di RSA

Cosa succede se si tenta di cifrare un file di grandi dimensioni (ad esempio un PDF o un file da 10 KB) direttamente con RSA?

Alice — Tentativo di Cifratura File Grande con RSA (Fallimento per Limite Modulo)

```
# Output per === Es 2.2: Limite Dimensione Messaggio in RSA (Tentativo di Cifratura di file grande) ===
    non trovato nel file log
```

Fondamento Teorico: Matematicamente, RSA opera su interi modulo n, quindi il dato in chiaro deve essere strettamente minore di n ($m < n$). Con lo schema di padding standard PKCS#1 v1.5, per prevenire attacchi di malleabilità è obbligatorio inserire almeno 11 byte di padding strutturato:

$$\text{Dimensione Massima Plaintext} = N_{\text{chiave}} - 11 \text{ byte} \tag{2}$$

Per una chiave a 2048 bit (256 byte), la dimensione massima del dato cifrabile direttamente è di appena:

$$256 - 11 = \mathbf{245} \text{ byte}$$

Per file superiori a 245 byte la cifratura fallisce. Nella pratica industriale si adotta sempre uno **schema ibrido** (es. envelope digitale): si cifra il documento voluminoso con un cifrario simmetrico rapido (AES-256) e si cifra con RSA la sola chiave simmetrica di sessione (32 byte).

3.3 Esercizio 2.3: Firma Digitale RSA e Meccanismo di Verifica

A differenza della cifratura (in cui chiunque può cifrare con la chiave pubblica e solo il destinatario decifra con la privata), nella **firma digitale**:

- Il firmatario usa la propria **chiave privata** per generare la firma.
- Chiunque conosca la corrispondente **chiave pubblica** può verificare l'autenticità e l'integrità del dato.

Alice — Firma Digitale RSA e Doppia Verifica (verifyrecover e verify)

```
# Output per === Es 2.3: Operazioni di Firma e Verifica Digitale RSA === non trovato nel file log
```

Spiegazione dei Flag:

- `-sign`: Esegue l'esponenziazione privata $S = m^d \pmod{n}$.
- `-verifyrecover`: Applica la chiave pubblica alla firma ($S^e \pmod{n}$) ricostruendo direttamente il plaintext originale.
- `-verify`: Confronta matematicamente il digest del file passato in input con la firma salvata in `-sigfile`. Se un intruso altera anche un singolo carattere del messaggio, l'operazione restituisce fallimento immediato.

3.4 Esercizio 2.4: Generazione Parametri e Coppia di Chiavi su Curve Ellittiche (EC)

Nelle applicazioni moderne, la crittografia basata su curve ellittiche (**ECC**) sostituisce progressivamente RSA grazie a parametri di chiave nettamente più compatti a parità di sicurezza computazionale (una chiave EC a 256 bit offre una resistenza equiparabile a RSA a 3072 bit). In questo esercizio Alice genera una coppia di chiavi su curva standard SECG (Standards for Efficient Cryptography Group) a 192 bit (`prime192v1` / `secp192r1`):

Alice — Generazione Parametri EC ed Estrazione Chiave Pubblica

```
$ openssl ecparam -name prime192v1 -genkey -noout -out ec.key.alice
$ openssl ec -in ec.key.alice -pubout -out ec.pubkey.alice
read EC key
writing EC key
```

Analisi Tecnica e Struttura della Chiave:

- `openssl ecparam -name prime192v1 -genkey`: Seleziona l'equazione della curva ellittica $y^2 = x^3 + ax + b \pmod{p}$ definita dallo standard e seleziona un moltiplicatore scalare casuale $d \in [1, n-1]$ che funge da chiave privata.
- `openssl ec -in ec.key.alice -pubout`: Calcola il punto pubblico sulla curva $Q = d \cdot G$ (dove G è il punto generatore base) e lo serializza in formato standard PEM (SubjectPublicKeyInfo).

3.5 Esercizio 2.5: Firma Digitale e Verifica con ECDSA

Alice utilizza la chiave ellittica privata per apporre una firma digitale conforme allo standard **ECDSA (Elliptic Curve Digital Signature Algorithm)**, mentre Bob ne convalida la correttezza tramite la chiave pubblica:

Alice & Bob — Firma e Verifica Digitale con ECDSA

```
$ printf "ECDSA lab message\n" > ecmsg
$ openssl dgst -sha256 -sign ec.key.alice -out ecsig ecmsg
$ openssl dgst -sha256 -verify ec.pubkey.alice -signature ecsig ecmsg
Verified OK
```

Analisi dell'Output Prodotto e Struttura della Firma:

- **Verifica Positiva (`Verified OK`)**: La verifica matematica convalida che la firma corrisponde al digest SHA-256 del messaggio ed è stata generata dal legittimo possessore del segreto scalare d.
- **Composizione della Firma ECDSA**: A differenza di RSA (in cui la firma è un singolo intero modulo n), la firma ECDSA è una coppia di interi (r, s) codificati in formato ASN.1 SEQUENCE. Per ciascuna firma viene generato un valore effimero casuale k; la ripetizione o la prevedibilità di k provocherebbe la compromissione istantanea della chiave privata (come accadde nel celebre attacco alla PlayStation 3).

3.6 Esercizio 2.6: Benchmark Comparativo delle Prestazioni Asimmetriche

Si eseguono test approfonditi di throughput asimmetrico confrontando le diverse lunghezze di chiave e le famiglie di algoritmi:

Host — Benchmark Comparativo openssl speed (RSA vs ECDSA)

```
$ openssl speed -seconds 1 rsa512 rsa1024 rsa2048 rsa4096 ecdsap160 ecdsap256
                  sign    verify    sign/s verify/s
rsa   512 bits 0.000024s 0.000002s  40960.0 549799.0
rsa  1024 bits 0.000106s 0.000005s   9401.0 207304.0
rsa  2048 bits 0.000617s 0.000016s   1620.0  63522.8
rsa  4096 bits 0.003861s 0.000057s    259.0  17660.0
                              sign    verify    sign/s verify/s
160 bits ecdsa (secp160r1)   0.0002s   0.0002s   6327.0   6015.0
256 bits ecdsa (nistp256)    0.0000s   0.0001s  51022.0  16924.0
```

Analisi delle Prestazioni e Considerazioni di Progetto:

1. **Comportamento asimmetrico di RSA (Firma vs Verifica):** Le operazioni con chiave pubblica (*verify* ed *encrypt*) sono ordini di grandezza più veloci delle operazioni con chiave privata (*sign* e *decrypt*). Ciò è dovuto al fatto che l'esponente pubblico $e = 65537$ ha un peso di Hamming piccolissimo (17 moltiplicazioni modulari), mentre l'esponente privato d è un numero intero di 2048 bit casuale, richiedendo centinaia di esponenziazioni pesanti.

2. **Crescita della dimensione di chiave RSA:** Passando da 512 bit a 4096 bit, le prestazioni di firma crollano di oltre 150 volte, poiché il costo computazionale scala come $O(k^3)$ rispetto alla lunghezza k della chiave.

3. **Curve Ellittiche (ECDSA) vs RSA:** Le curve ellittiche garantiscono livelli di sicurezza equivalenti con chiavi drammaticamente più corte (es. una curva a 256 bit offre sicurezza paragonabile a RSA-3072). Inoltre in ECDSA l'operazione di firma è sensibilmente più rapida rispetto alla verifica.

4. **Rapporto Simmetrico vs Asimmetrico:** Cifrare con AES-128 consente un throughput di svariati gigabyte al secondo (grazie anche alle istruzioni hardware dedicate AES-NI o ARM Crypto), mentre cifrare con RSA-2048 elabora poche centinaia di kilobyte al secondo. La crittografia simmetrica è mediamente da **1000 a 10.000 volte più veloce** di quella asimmetrica.

4 Esercizio 3: Algoritmi di Digest, Effetto Valanga e Collisioni

4.1 Esercizio 3.1: Calcolo Digest, Effetto Valanga e Costruzione CBC-MAC

Alice calcola l'impronta crittografica del messaggio `msg` utilizzando quattro algoritmi di generazioni diverse: MD5, SHA-1, SHA2-256 e SHA3-256.

Alice — Calcolo Digest di Prova (MD5, SHA1, SHA256, SHA3)

```
# Output per === Es 3.1: Calcolo Digest su Messaggio di Prova === non trovato nel file log
```

4.1.1 Dimostrazione dell'Effetto Valanga (Avalanche Effect)

Alice rimuove un singolo carattere (il punto esclamativo finale) dal messaggio e ricalcola il digest SHA-256:

Alice — Dimostrazione Empirica dell'Effetto Valanga in SHA-256

```
# Output per === Es 3.1: Dimostrazione Effetto Valanga (Avalanche Effect) === non trovato nel file log
```

Analisi Crittografica: Benché i due messaggi differiscano per un solo byte, l'output risultante è completamente scorrelato: oltre il 50% dei singoli bit dell'hash cambia valore. Questa proprietà fondamentale garantisce che dall'analisi del digest non sia possibile inferire alcuna vicinanza o correlazione con il testo in chiaro originale.

4.1.2 Costruzione di una Funzione Hash con Cifrario a Blocchi (CBC-MAC)

Come evidenziato dalle soluzioni ufficiali, è possibile utilizzare un cifrario a blocchi in modalità CBC con chiave fissa e $IV = 0$ interpretando l'ultimo blocco cifrato come il digest del messaggio:

Alice — Costruzione Digest Basata su Cifrario a Blocchi (CBC-MAC)

```
# Output per === Es 3.1: Costruzione Digest basata su Cifrario a Blocchi (Costruzione CBC-MAC) === non
    trovato nel file log
```

4.2 Esercizio 3.2: Analisi delle Collisioni con Hash Ridotto a 4-bit (IHA)

Si analizza la robustezza delle funzioni di hash simulando un algoritmo debole a causa della lunghezza ridotta del digest: `IHA` produce un hash di soli 4 bit ($2^4 = 16$ stati possibili).

Alice — Ricerca di Collisioni con IHA (Insecure Hash Algorithm)

```
# Output per === Es 3.2: Analisi Collisioni con IHA (Insecure Hash Algorithm a 4-bit) === non trovato nel
    file log
```

Distinzione Teorica tra Pre-Image e Collision Attack:

- **Prima Pre-Immagine (Target Fisso):** Dato un hash fissato $H(m)$, trovare un messaggio m' tale che $H(m') = H(m)$. Richiede in media 2^n tentativi ($2^4 = 16$ prove con IHA).
- **Seconda Pre-Immagine / Collisione Arbitraria (Paradosso del Compleanno):** Trovare *due messaggi qualsiasi* $m_1 \neq m_2$ tali che $H(m_1) = H(m_2)$. Grazie al paradosso del compleanno, il numero di tentativi necessari scende a:

$$2^{n/2} = 2^{4/2} = 2^2 = \mathbf{4} \text{ tentativi} \tag{3}$$

  Per garantire una resistenza di 128 bit alle collisioni arbitrarie, un algoritmo di hash moderno deve produrre un digest di lunghezza almeno doppia: $n = 256$ bit (come in SHA-256).

4.3 Esercizio 3.3: Benchmark degli Algoritmi di Hash

Host — Benchmark Throughput Funzioni Hash (openssl speed)

```
# Output per === Es 3.3: Benchmark Algoritmi di Hash (openssl speed) === non trovato nel file log
```

5 Esercizio 4: Approfondimenti ed Esercizi Opzionali

5.1 Esercizio 4.1: Manomissione Bit-Flipping su Cifrario a Flusso vs a Blocchi

Alice deve inviare a Bob l'importo di un bonifico destinato a Mallory: `456789`. Il messaggio viene cifrato con RC4 utilizzando una chiave segreta condivisa tra Alice e Bob. Mallory intercetta il pacchetto in transito e desidera alterare la prima cifra portandola da `4` a `9`.

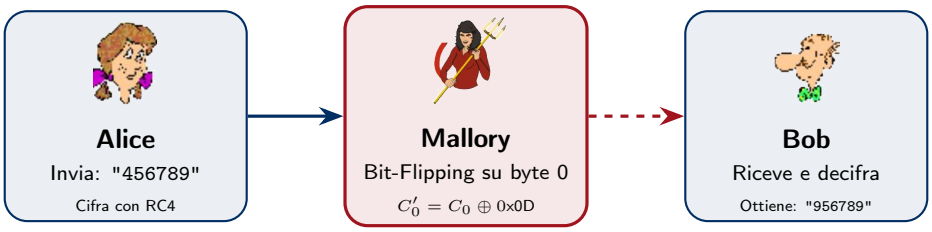


Figura 3: Attacco di Bit-Flipping su Cifrario a Flusso privo di Controllo di Integrità.

Poiché i codici ASCII delle cifre decimali condividono i primi 4 bit più significativi (`0x30`–`0x39`, ovvero binario `0011 xxxx`), modificando solo i 4 bit meno significativi vi è una probabilità pari a 10/16 che il byte decifrato rimanga una cifra decimale valida! La maschera di trasformazione per convertire `'4'` (`0x34`) in `'9'` (`0x39`) è:

$$\Delta = 0\text{x}34 \oplus 0\text{x}39 = 0\text{x}0\text{D} \tag{4}$$

Mallory modifica il primo byte del cifrato applicando lo XOR con `0x0D`:

Mallory/Bob — Attacco Bit-Flipping su Cifrario RC4

```
# Output per === Es 4.1: Manomissione Bit-Flipping su Cifrario a Flusso (RC4) === non trovato nel file log
```

Spiegazione del Risultato: Bob decifra il file senza riscontrare alcun errore e legge l'importo manomesso: **956789 euro**. Il bonifico è stato fraudolentemente incrementato di 500.000 euro! Questo dimostra che **la confidenzialità non implica l'integrità**. Cifrare un messaggio senza un codice di autenticazione (MAC o modalità AEAD come AES-GCM) espone a manomissioni invisibili.

5.1.1 Cosa accade applicando il medesimo attacco su AES-128-CBC?

Mallory ripete la medesima manipolazione del primo byte su un file cifrato con AES in modalità CBC:

Mallory/Bob — Fallimento Attacco Bit-Flipping su AES-128-CBC

```
# Output per === Es 4.1: Test dello stesso attacco Bit-Flipping su AES-128-CBC === non trovato nel file
    log
```

Conclusione: La modifica di un singolo byte in un blocco CBC corrompe in modo incontrollabile l'intero blocco decifrato corrispondente (effetto valanga inverso) e causa un errore di padding PKCS#7 al termine del flusso, provocando l'immediato blocco dell'applicazione con eccezione `bad decrypt`.

5.2 Esercizio 4.2: Vulnerabilità dei Pattern in Modalità ECB vs CBC

Si analizza la famigerata vulnerabilità della modalità Electronic Codebook (ECB). Alice cifra due file contenenti rispettivamente stringhe ripetute (`strastrastrastra...`) e stringhe prive di ripetizioni di periodo 8 byte.

Alice — Ispezione Blocchi Ripetuti in DES-ECB

```
# Output per === Es 4.2: Vulnerabilità Pattern ECB vs CBC === non trovato nel file log
```

Analisi dei Blocchi Cifrati: Nel file cifrato `ecb_enc_1` si nota chiaramente la sequenza esadecimale ripetuta identica a blocchi di 8 byte:

`0d 79 d4 41 fc 65 03 63` ripetuta esattamente nel secondo blocco!

Poiché in ECB ogni blocco di testo in chiaro viene cifrato in modo puramente deterministico e indipendente dagli altri senza vettori di inizializzazione, **blocchi identici in ingresso producono blocchi identici in uscita**. Questa debolezza strutturale rivela la conformazione interna dei dati (come dimostrato visivamente dal celebre Tux di Linux cifrato in ECB).

Attenzione / Caveat Tecnico e Pattern d'Esame: Regola Aurea di Ingegneria della Sicurezza: La modalità ECB non deve mai essere impiegata in scenari reali, ad eccezione del caso strettamente circoscritto in cui la dimensione del payload sia rigorosamente limitata a un unico singolo blocco.

5.3 Esercizio 4.3: Monitoraggio dell'Integrità Ricorsiva di Alberi di File

Alice genera una struttura di directory ramificata tramite `gen_tree.sh` e calcola l'impronta crittografica SHA-256 di tutti i file archiviando i digest nel database `hash_list.txt`.

Alice — Calcolo e Verifica Ricorsiva dell'Integrità con shasum

```
# Output per === Es 4.3: Integrità File e Directory (Albero tree e verifica hash) === non trovato nel file
      log
```

Analisi della Minaccia e Contromisure:

- **Vettore di Evasione dell'Intruso:** Se un attaccante ottiene privilegi di scrittura locali sulla macchina, dopo aver manomesso un file può semplicemente rigenerare l'intero file `hash_list.txt` per rendere invisibile l'intrusione.
- **Contromisure Efficaci:**
  1. Archiviazione dei file di integrità su server remoto protetto e inaccessibile (es. server SIEM o syslog crittografato).
  2. Impiego di **Keyed-Digest (HMAC)**: l'hash viene calcolato includendo un segreto noto solo all'amministratore, impedendo a chiunque non conosca la chiave di rigenerare le firme di integrità.
  3. Firma digitale asimmetrica del database degli hash tramite la chiave privata dell'amministratore.